

Guía de Aprendizaje: Manejo de Archivos en C

Malak Sanchez

Monitorías de Sistemas Operativos

1. Motivación

Los datos en memoria RAM son volátiles: desaparecen al apagar el equipo. En Sistemas Operativos, necesitamos persistencia para guardar configuraciones, logs o bases de datos. Para ello usamos archivos.

2. Idea Fundamental

Un archivo es un flujo (*stream*) de datos almacenados en un dispositivo de almacenamiento. C maneja archivos mediante una estructura especial llamada FILE.

3. Sintaxis Básica

```
FILE *file_ptr = fopen("filename.txt", "mode");
fclose(file_ptr);
```

4. Modos de Apertura

El segundo argumento de `fopen` define qué podemos hacer con el archivo. Aquí los más comunes:

Modo	Descripción	¿Crea archivo?	¿Sobrescribe?
"r"	Lectura: El archivo debe existir.	No	No
"w"	Escritura: Crea el archivo o lo vacía si existe.	Sí	Sí
"a"	Anexar: Escribe al final del archivo.	Sí	No
"r+"	Lectura/Escritura: El archivo debe existir.	No	No
"w+"	Lectura/Escritura: Crea o sobrescribe.	Sí	Sí
"a+"	Lectura/Anexar: Lee y escribe al final.	Sí	No

- **Modo Binario:** Si trabajas con archivos que no son de texto (imágenes, ejecutables), añade una "b" al modo (ej: "rb", "wb").

5. Ejemplo Mínimo Funcional

```
1 #include <stdio.h>
2
3 int main() {
4     // Abrir para escribir (si existe, se borra el contenido
5     // anterior)
6     FILE *f = fopen("output.txt", "w");
7     if(f == NULL){
8         perror("Error opening file");
9         return 1;
10    }
11
12    fprintf(f, "Operating Systems 2026-I\n");
13
14    fclose(f);
15    return 0;
16 }
```

6. Explicación Paso a Paso

1. Se declara un puntero de tipo FILE.
2. fopen abre el archivo. El modo "w" es para escritura (*write*).
3. Se verifica que el archivo se haya abierto correctamente (`f != NULL`).
4. fprintf escribe datos en el archivo, similar a printf.
5. fclose cierra el flujo y asegura que los datos se guarden en el disco.

7. Qué ocurre en memoria

El Sistema Operativo mantiene una tabla de descriptores de archivos para cada proceso. El puntero FILE apunta a una estructura que contiene el estado del flujo (posición actual, buffer, etc.).

8. Patrones comunes de uso

Lectura línea por línea:

```
1 char buffer[100];
2 while(fgets(buffer, 100, f) != NULL){
3     printf("%s", buffer);
4 }
```

9. Errores Comunes

- **No cerrar el archivo:** Puede causar pérdida de datos o agotar el límite de archivos abiertos del sistema operativo.
- **Modos incorrectos:** Intentar escribir en un archivo abierto como "**r**" (lectura).

10. Buenas Prácticas

- Siempre verificar si `fopen` tuvo éxito (usar `perror` ayuda a debuggear).
- Cerrar los archivos apenas se termine de usarlos.

11. Ejercicios

1. Cree un programa que lea un archivo llamado `data.txt` e imprima cuántas líneas tiene.
2. Escriba un programa que pida el nombre del usuario y lo guarde en un archivo `config.log`.